PROJECT REPORT

ON

COMPOSITE BEHAVIORAL MODELING FOR IDENTITY THEFT DETECTION IN ONLINE SOCIAL NETWORKS

SUBMITTED IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF

DIPLOMA IN ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING BY

(23054-AI-010) C.PREMNADH
(23054-AI-018) K.SAI RAJ
(23054-AI-023) KARA KARTHIKEYA
(23054-AI-026) MANTHRI HARI KUMAR
(23054-AI-028) V.YEKESHWAR NAIK
(23054-AI-045) SHAIK MOHAMMED IMAADH
(23054-AI-046) PATIMEEDI KRISHNA
(23054-AI-051) YEJJU SATHYASAI
(23054-AI-053) KATA TEJESHWAR
(23054-AI-062) KOGANTI SAI CAHRAN
(23054-AI-064) MARTHA NITHIN KUMAR
(23054-AI-065) L.RAMDAS

UNDER THE GUIDANCE OF SRI KM.ARAVIND KUMAR,SL/EC

GOVERNMENT INSTITUTE OF ELECTRONICS, SECUNDERABAD

(Approved by TS SBTET,Hyderabad)

East Marredpally,Secunderabad – 500026

2023-2026

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

CERTIFICATE

This is to certify that the project entitled "UNDER WATER TRASH DETECTION SYSTEM" has been submitted by the following students:

(23054-Ai-010) C.Premnadh, (23054-Ai-018) K.Sai Raj, (23054-Ai-023) Kara Karthikeya, (23054-Ai-026) Manthri Hari Kumar, (23054-Ai-028) V.Yekeshwar Naik, (23054-Ai-045)Shaik Mohammed Imaadh,(23054-Ai-046) Patimeedi Krishna, (23054-Ai-051) Yejju Sathyasai, (23054-Ai-053) Kata Tejeshwar, (23054-Ai-062) Koganti Sai Cahran, (23054-Ai-064) Martha Nithin Kumar, (23054-Ai-065) L.Ramdas This project is submitted in partial fulfillment of the requirements for the award of the Diploma in Artificial Intelligence and Machine Learning from the Government Institute of
Electronics, East Marredpally, Secunderabad. The results embodied in this project have not been submitted to any other institution for the award of any degree or diploma.

GUIDE **HEAD OF DEPARTMENT**

SRI KM.Aravind Kumar, N.SrinivasRao

SL/EC.

EXTERNAL EXAMINER **TRAINER**

GOVERNMENT INSTITUTE OF ELECTRONICS, SECUNDERABAD

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

DECLARATION

We hereby declare that the project entitled " UNDER WATER TRASH DETECTION SYSTEM " is the work done during the period from DEC 2025 to MAY 2026 and is submitted in partial fulfillment of the requirements for the award of Diploma in Artificial Intelligence and Machine Learning from Govt.
Institute of Electronics, Secunderabad. The results embodied in this project have not been submitted to any other university or institution for the award of any degree or diploma.

(23054-AI-010) C.PREMNADH

(23054-AI-018) K.SAI RAJ

(23054-AI-023) KARA KARTHIKEYA

(23054-AI-026) MANTHRI HARI KUMAR

(23054-AI-028) V.YEKESHWAR NAIK

(23054-AI-045) SHAIK MOHAMMED IMAADH

(23054-AI-046) PATIMEEDI KRISHNA

(23054-AI-051) YEJJU SATHYASAI

(23054-AI-053) KATA TEJESHWAR

(23054-AI-062) KOGANTI SAI CAHRAN

(23054-AI-064) MARTHA NITHIN KUMAR

(23054-AI-065) L.RAMDAS

DEPARTMENT OF

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

ACKNOWLEDGEMENT

With the great pleasure and deep sense of gratitude, we acknowledge the contribution of all the faculty for the successful completion of this project.

We deem to be great honour to thank Mrs. **B.RAM MOHAN REDDY** sir, principal of **GOVERNMENT INSTITUTE OF ELECTRONICS** who is an axle behind our studies and for having given me a chance to pursue my diploma course in this esteemed institution.

Our thanks are due to Mr. **N. SRINIVAS RAO** sir, Head of the department for his excellent, expert support rendered to me during the tenure in the institute.

We would like to express our gratitude and appreciation to all the people who gave us the possibility to complete this report.

Finally, I would like to thank **SRI KM.ARAVIND KUMAR** sir,SL/EC, project head, who gave us all the required support to understand all the technical aspects both in theory and practice during the project.

(23054-AI-010) C.PREMNADH

(23054-AI-018) K.SAI RAJ

(23054-AI-023) KARA KARTHIKEYA

(23054-AI-026) MANTHRI HARI KUMAR

(23054-AI-028) V.YEKESHWAR NAIK

(23054-AI-045) SHAIK MOHAMMED IMAADH

(23054-AI-046) PATIMEEDI KRISHNA

(23054-AI-051) YEJJU SATHYASAI

(23054-AI-053) KATA TEJESHWAR

(23054-AI-062) KOGANTI SAI CAHRAN

(23054-AI-064) MARTHA NITHIN KUMAR

(23054-AI-065) L.RAMDAS

ABSTRACT

In this work, we aim at building a bridge from coarse behavioral data to an effective, quick-response, and robust behavioral model for online identity theft detection. We concentrate on this issue in online social networks (OSNs) where users usually have composite behavioral records, consisting of multidimensional low-quality data, e.g., offline check-ins and online user-generated content (UGC). As an insightful result, we validate that there is a complementary effect among different dimensions of records for modeling users' behavioral patterns. To deeply exploit such a complementary effect, we propose a *joint* (instead of *fused*) model to capture both online and offline features of a user's composite behavior. We evaluate the proposed joint model by comparing it with typical models and their fused model on two real-world datasets: Foursquare and Yelp. The experimental results show that our model outperforms the existing ones, with the area under the receiver operating characteristic curve (AUC) values 0.956 in Foursquare and 0.947 in Yelp, respectively. Particularly, the *recall* (true positive rate) can reach up to 65.3% in Foursquare and 72.2% in Yelp with the corresponding *disturbance rate* (false-positive rate) below 1%. It is worth mentioning that these performances can be achieved by examining only one composite behavior, which guarantees the low response latency of our method. This study would give the cybersecurity community new insights into whether and how real-time online identity authentication can be improved via modeling users' composite behavioral patterns.

INDEX

1. Introduction

2. System Study

- Existing System vs. Proposed System
- Feasibility Study

3. Software Environment

- Python Environment Setup

4. System Architecture

- Architecture Overview
- System Modules
- Algorithms

5. System Design (UML Diagrams)

- Use Case Diagram
- Class Diagram
- Sequence Diagram
- Data Flow Diagram
- Flow Chart Diagram

6. System Testing

- Testing Methodologies
- Test Cases and Results

7. Conclusion

8. References

1. INTRODUCTION

WITH the rapid development of the Internet, more and more affairs, e.g., mailing [1], health caring [2], shopping [3], booking hotels, and purchasing tickets, are handled online [4].Meanwhile, the Internet also brings sundry potential risks of invasions, such as losing financial information [5], identity theft [6], and privacy leakage [3]. Online accounts serve as the agents of users in the cyber world. Online identity theft is a typical online crime which is the deliberate use of another person's account [7], usually as a method to gain a financial advantage or obtain credit and other benefits in another person's name. As a matter of fact, compromised accounts are usually the portals of most cybercrimes [1], such as blackmail [5], fraud [8], and spam [9], [10]. Thus, identity theft detection is essential to guarantee users' security in the cyber world.

Traditional identity authentication methods are mostly based on access control schemes, e.g., passwords and tokens [11], [12]. But users have some overheads in managing dedicated passwords or tokens. Accordingly, the biometric identification [13]–[15] is delicately introduced to start the era of password-free. However, some disadvantages make these access control schemes still incapable of being effective in real-time online services [16], [17].

1) They are not *nonintrusive*. Users have to spend extra time in the authentication.

2) They are not *continuous*. The defending system will fail to take further protection once the access control is broken.

Behavior-based suspicious account detection [16], [18], [19] is a highly anticipated solution to pursue a nonintrusive and continuous identity authentication for online services. It depends on capturing users' suspicious behavior patterns to discriminate the suspicious accounts. The problem can be divided into two categories: fake/sybil account detection [20] and compromised account detection [21]. The fake/Sybil account's behaviors usually do not conform to the behavioral pattern of the majority. Meantime, the compromised account usually behaves in a pattern that does not conform to its previous one, even behaves like fake/sybil accounts. It can be solved by capturing *mutations* of users' behavioral patterns.Comparing with detecting compromised accounts, detecting fake/sybil accounts is relatively easy since the latter's behaviors are generally more detectable than the former's. It has been extensively studied and can be realized by various population-level approaches, e.g., clustering [22], [23], classification [5], [24]–[26] and statistical or empirical rules [8], [27], [28]. Thus, we *only* focus on the compromised account detection, commonly called *identity theft detection*, based on individual-level behavioral models.

Recently, researchers have proposed the individual-level identity theft detection methods by using suspicious behavior detection [9], [29]–[35]. The efficacy of these methods significantly depends on the

sufficiency of behavior records. They are usually suffering from the low-quality of behavior records due to data collecting limitations or some privacy issues [3]. In particular, when a method only utilizes a specific dimension of behavioral data, the efficacy damaged by poor data is possibly enlarged and the scope of application is limited. Unfortunately, many existing works just concentrate on a specific dimension of users' behavior, such as keystroke [29], click stream [32], [36], touch-interaction [37], and user generated content (UGC) [9], [33], [34], [38].

In this article, we propose an approach to detect identity theft by using multidimensional behavioral records which are possibly insufficient in each dimension. According to such characteristics, we choose the online social network (OSN) as a typical scenario where most users' behaviors are coarsely recorded [39]. In the Internet era, users' behaviors are composited by offline behaviors, online behaviors, social behaviors, and perceptual/cognitive behaviors. The behavioral data can be collected in many applications, such as offline check-ins in location-based services (LBSs), online tips-posting in instant messaging services, and social relationship-making in online social services. Accordingly, we design our method based on users' composite behaviors by these categories.

In OSNs, user behavioral data that can be used for online identity theft detection are often too low-quality or restricted to build qualified behavioral models due to the difficulty of data collection, the requirement of user privacy, and the fact that some users have a few several behavioral records. We devote ourselves to proving that a high-quality (effective, quick response, and robust) behavioral model can be obtained by integrally using multidimensional behavioral data, even though the data is extremely insufficient in each dimension.

Generally, there are two paradigms to integrate behavioral data: the *fused* and *joint* manners. Fused models are a relatively simple and straightforward kind of composite behavior models (CBMs). They first capture features in each behavior space and then make a comprehensive metric based on these features in different dimensions. With the possible complementary effect among different behavior spaces, they can act as a feasible solution for integration [7], [17]. However, the identification efficacy can be further improved, since fused models neglect potential links among different spaces of behaviors. We take an example where a person posted a picture in an OSN when he/she visited a park. If this composite behavior is simply separated into two independent parts: he/she once posted a picture and he/she once visited a park, the difficulty in relocating him/her from a group of users is possibly increased, since there are more users satisfy these two simple conditions comparing to the original condition. In contrast, the joint model can sufficiently exploit the correlations between behaviors in different dimensions, then increases the certainty of users'

behavior patterns, which contributes to a better identification efficacy. The underlying logic for the difference between the joint and fused models can be also explained by the well-known *Chain Rule for Entropy* [40], which indicates that the entropy of multiple simultaneous events is no more than the sum of the entropies of each individual event, and are equal if the events are independent. It shows that the joint behavior has lower uncertainty comparing to the sum of the uncertainty in each component [41].

Therefore, to fully utilize potential information in composite behaviors for user profiling, we propose a *joint* model, specifically, a joint probabilistic generative model based on Bayesian networks called CBM. It offers a composition of the typical features in two different behavior spaces: check-in location in offline behavior space and UGC in online behavior space. Considering the composite behavior of a user, we assume that the generative mechanism is as follows. When a user plans to visit a venue and simultaneously post tips online, he/she subconsciously selects a specific behavioral pattern according to his/her behavioral distribution. Then, he/she comes up with a topic and a targeted venue based on the present pattern's topic and venue distributions, respectively. Finally, his/her comments are generated following the corresponding topic-word distribution. To estimate the parameters of the mentioned distributions, we adopt the collapsed Gibbs sampling [42].

Based on the joint model CBM, for each composite behavior, denoted by a triple-tuple *(u, v,D)*, we can calculate the chance of user *u* visiting venue *v* and posting a tip online with a set of words *D*. Taking into account different levels of activity of different users, we devise a *relative anomalous score Sr* to measure the occurrence rate of each composite behavior *(u, v,D)*. By these approaches, we finally realize real-time detection (i.e., judging by only one composite behavior) for identity theft suspects.

We evaluate our joint model by comparing it with three typical models and their fused model [17] on two real-world OSN datasets: Foursquare [43] and Yelp [44]. We adopt the area under the receiver operating characteristic curve (AUC) as the detection efficacy. Particularly, the *recall* [true positive rate (TPR)] reaches up to 65.3% in Foursquare and 72.2% in Yelp, respectively, with the corresponding *disturbance rate* [false-positive rate (FPR)] below 1%, while the fused model can only achieve 60.8% and 60.4% in the same condition, respectively. Note that this performance can be achieved by examining only one composite behavior per authentication, which guarantees the low response latency of our detection method. As an insightful result, we learn that the complementary effect does exist among different dimensions of low-quality records for modeling users' behaviors.

The main contributions are summarized into three folds.

1) We propose a joint model, CBM, to capture both online and offline features of a user's composite behavior to fully exploit coarse behavioral data.

2) We devise a relative anomalous score *Sr* to measure the occurrence rate of each composite behavior for realizing real-time identity theft detection.

3) We perform experiments on two real-world datasets to demonstrate the effectiveness of CBM. The results show that our model outperforms the existing models and has the low response latency.

The rest of this article is organized as follows. We give an overview of our solution in Section II. Then, we present our method in Section III, and make the validation in Section IV. We provide a literature review in Section V. Finally, we draw conclusions in Section VI.

2. SYSTEM STUDY

2.1 Existing Sytem vs Proposed System

EXISTING SYSTEM

Sitova *et al.* [53] introduced hand movement, orientation, and grasp (HMOG), a set of behavioral features to continuously authenticate smartphone users. Rajoub and Zwiggelaar [15] used thermal imaging to monitor the periorbital region's thermal variations and test whether it can offer a discriminative signature for detecting deception. However, these biometric technologies usually require expensive hardware devices which makes it inconvenient and difficult to popularize.

Abouelenien *et al.* [30] explored a multimodal deception detection approach that relied on a novel dataset of 149 multimodal recordings, and integrated multiple physiological, linguistic, and thermal features. These works indicated that users' behavior patterns can represent their identities. Many studies turn to utilize users' behavior patterns for identifications. Behavior-based methods were born at the right
moment, which plays important roles in a wide range of tasks including preventing and detecting identity theft. Typically, behavior-based user identification includes two phases: user profiling and user identifying.

User profiling is a process to characterize a user with his/her history behavioral data. Some works focus on statistical characteristics, such as the mean, variance, median, or frequency of a variable, to establish the user profile. Naini *et al.* [55] studied the task of identifying the users by matching the histograms of their data in the anonymous dataset with the histograms from the original dataset. But it mainly relied on experts' experience since different cases usually have different characteristics.
Egele *et al.* [7] proposed a behavior-based method to identify compromises of individual high-profile accounts. However, it required high-profile accounts which were difficult to obtain.
Other researchers discovered other features, such as tracing patterns, topic and spatial distributions, to describe user identity. Ruan *et al.* [32] conducted a study on online user behavior by collecting and analyzing user clickstreams of a well-known OSN. Lesaege *et al.* [31] developed a topic model extending the LDA to identify the active users. Viswanath *et al.* [56] presented a technique based on principal component analysis (PCA) that accurately modeled the "like" behavior of normal users in Facebook and identified significant deviations from it as anomalous behaviors. Zaeem *et al.* [33] proposed an approach that involved the novel collection of online news stories and reports on the topic of identity theft. Lichman and Smyth [48] proposed MKDE model to accurately characterize and predict the spatial pattern of an individual's events.

Tsikerdekis and Zeadally [57] presented a detection method based on nonverbal behavior for identity deception, which can be applied to many types of social media. These methods above mainly concentrated on a specific dimension of the composite behavior and seldom thought about utilizing multidimensional behavior data. Sekara *et al.* [58] explored the complex interaction between social and geospatial behavior and demonstrated that social behavior can be predicted with high precision. It indicated that composite behavior features can identify one's
identity.

Yin *et al.* [42] proposed a probabilistic generative model combining the use of spatiotemporal data and semantic information to predict user's behavior. Nilizadeh *et al.* [49] presented POISED, a system that leverages the differences in propagation between benign and malicious messages on social networks to identify spam and other unwanted content. These studies implied that composite behavior features are possibly helpful for user identification.

Disadvantages

1) LDA model performs poorly in both datasets which may indicate its performance is strongly sensitive to the data quality.
2) CF-KDE and LDA model performs not well in Yelp dataset comparing to Foursquare dataset, but the fused model [17] observes a surprising reversion.
3) The joint model based on *relative anomalous score Sr* outperforms the model based on *logarithmic anomalous score Sl* .
4) The joint model (i.e., JOINT-SR, the joint model in the following content of the system all refer to the joint model based on *Sr*) is indeed superior to the fused model.

Proposed System

In this article, we propose an approach to detect identity theft by using multidimensional behavioral records which are possibly insufficient in each dimension. According to such characteristics, we choose the online social network (OSN) as a typical scenario where most users' behaviors are coarsely recorded [39]. In the Internet era, users' behaviors are composited by offline behaviors, online behaviors, social behaviors, and perceptual/cognitive behaviors. The behavioral data can be collected in many applications, such as offline

check-ins in location-based services (LBSs), online tips-posting in instant messaging services, and social relationship-making in online social services. Accordingly, we design our method based on users' composite behaviors by these categories.

In OSNs, user behavioral data that can be used for online identity theft detection are often too low-quality or restricted to build qualified behavioral models due to the difficulty of data collection, the requirement of user privacy, and the fact that some users have a few several behavioral records. We devote ourselves to proving that a high-quality (effective, quickresponse, and robust) behavioral model can be obtained by integrally using multidimensional behavioral data, even though the data is extremely insufficient in each dimension.

Advantages

1) We propose a joint model, CBM, to capture both online and offline features of a user's composite behavior to fully exploit coarse behavioral data.

2) We devise a relative anomalous score Sr to measure the occurrence rate of each composite behavior for realizing real-time identity theft detection.

3) We perform experiments on two real-world datasets to demonstrate the effectiveness of CBM. The results show that our model outperforms the existing models and has the low response latency.

SYSTEM REQUIREMENTS

- H/W System Configuration:-
- Processor - Pentium –IV
- RAM - 4 GB (min)
- Hard Disk - 20 GB
- Key Board - Standard Windows Keyboard
- Mouse - Two or Three Button Mouse
- Monitor - SVGA

SOFTWARE REQUIREMENTS:

- Operating system : Windows 7 Ultimate.
- Coding Language : Python.
- Front-End : Python.

- Back-End : Django-ORM
- Designing : Html, css, javascript.
- Data Base : MySQL (WAMP Server).

2.2 FEASIBILITY STUDY

The feasibility of the project is analyzed in this phase and business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system is essential.

Three key considerations involved in the feasibility analysis are

- ECONOMICAL FEASIBILITY
- TECHNICAL FEASIBILITY
- SOCIAL FEASIBILITY

ECONOMICAL FEASIBILITY

This study is carried out to check the economic impact that the system will have on the organization. The amount of fund that the company can pour into the research and development of the system is limited. The expenditures must be justified. Thus the developed system as well within the budget and this was achieved because most of the technologies used are freely available. Only the customized products had to be purchased.

TECHNICAL FEASIBILITY

This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. This will lead to high demands on the available technical resources. This will lead to high demands being placed on the client. The developed system must have a modest requirement, as only minimal or null changes are required for implementing this system.

SOCIAL FEASIBILITY

The aspect of study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The user must not feel threatened by the system, instead must accept it as a necessity. The level of acceptance by the users solely depends on the methods that are employed to educate the user about the system and to make him familiar with it. His level of confidence must be raised so that he is also able to make some constructive criticism, which is welcomed, as he is the final user of the system.

3. SYSTEM ENVIRONMENT

3.1 PYTHON

Python is a **high-level, interpreted**, **interactive** and **object-oriented scripting language**. Python is designed to be highly readable. It uses English keywords frequently where as other languages use punctuation, and it has fewer syntactical constructions than other languages.

- **Python is Interpreted:** Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- **Python is Interactive:** You can actually sit at a Python prompt and interact with the interpreter directly to write your programs.
- **Python is Object-Oriented:** Python supports Object-Oriented style or technique of programming that encapsulates code within objects.
- **Python is a Beginner's Language:** Python is a great language for the beginner-level programmers and supports the development of a wide range of applications from simple text processing to WWW browsers to games.

3.2 History of Python

Python was developed by Guido van Rossum in the late eighties and early nineties at the National Research Institute for Mathematics and Computer Science in the Netherlands.

Python is derived from many other languages, including ABC, Modula-3, C, C++, Algol-68, SmallTalk, and Unix shell and other scripting languages.

Python is copyrighted. Like Perl, Python source code is now available under the GNU General Public License (GPL).

Python is now maintained by a core development team at the institute, although Guido van Rossum still holds a vital role in directing its progress.

3.3 Python Features

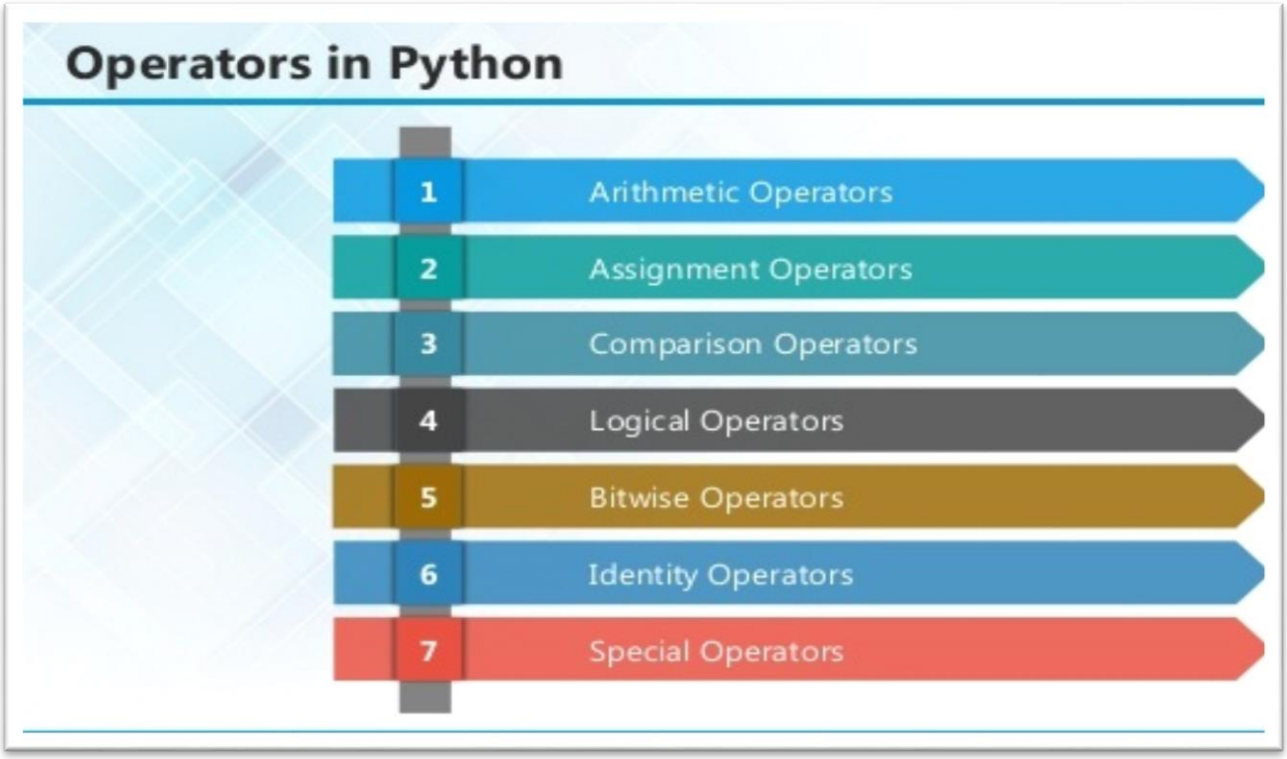
Python's features include:

- **Easy-to-learn:** Python has few keywords, simple structure, and a clearly defined syntax. This allows the student to pick up the language quickly.
- **Easy-to-read:** Python code is more clearly defined and visible to the eyes.
- **Easy-to-maintain:** Python's source code is fairly easy-to-maintain.
- **A broad standard library:** Python's bulk of the library is very portable and cross-platform compatible on UNIX, Windows, and Macintosh.

- **Interactive Mode:** Python has support for an interactive mode which allows interactive testing and debugging of snippets of code.
- **Portable:** Python can run on a wide variety of hardware platforms and has the same interface on all platforms.
- **Extendable:** You can add low-level modules to the Python interpreter. These modules enable programmers to add to or customize their tools to be more efficient.
- **Databases:** Python provides interfaces to all major commercial databases.
- **GUI Programming:** Python supports GUI applications that can be created and ported to many system calls, libraries and windows systems, such as Windows MFC, Macintosh, and the X Window system of Unix.
- **Scalable:** Python provides a better structure and support for large programs than shell scripting.

Python has a big list of good features:

- It supports functional and structured programming methods as well as OOP.
- It can be used as a scripting language or can be compiled to byte-code for building large applications.
- It provides very high-level dynamic data types and supports dynamic type checking.
- IT supports automatic garbage collection.
- It can be easily integrated with C, C++, COM, ActiveX, CORBA, and Java.



3.4 ARITHMETIC OPERATORS

Operator	Description	Example
+ Addition	Adds values on either side of the operator.	a + b = 30
- Subtraction	Subtracts right hand operand from left hand operand.	a – b = -10
* Multiplication	Multiplies values on either side of the operator	a * b = 200
/ Division	Divides left hand operand by right hand operand	b / a = 2
% Modulus	Divides left hand operand by right hand operand and returns remainder	b % a = 0

** Exponent	Performs exponential (power) calculation on operators	a**b =10 to the power 20
//	Floor Division - The division of operands where the result is the quotient in which the digits after the decimal point are removed. But if one of the operands is negative, the result is floored, i.e., rounded away from zero (towards negative infinity):	9//2 = 4 and 9.0//2.0 = 4.0, -11//3 = -4, -11.0//3 = -4.0

3.5 ASSIGNMENT OPERATOR

Operator	Description	Example
=	Assigns values from right side operands to left side operand	c = a + b assigns value of a + b into c
+= Add AND	It adds right operand to the left operand and assign the result to left operand	c += a is equivalent to c = c + a
-= Subtract AND	It subtracts right operand from the left operand and assign the result to left operand	c -= a is equivalent to c = c - a
*= Multiply AND	It multiplies right operand with the left operand and assign the result to left operand	c *= a is equivalent to c = c * a
/= Divide AND	It divides left operand with the right operand and assign the result to left operand	c /= a is equivalent to c = c /

		ac /= a is equivalent to c = c / a

%= Modulus AND	It takes modulus using two operands and assign the result to left operand	c %= a is equivalent to c = c % a
**= Exponent AND	Performs exponential (power) calculation on operators and assign value to the left operand	c **= a is equivalent to c = c ** a
//= Floor Division	It performs floor division on operators and assign value to the left operand	c //= a is equivalent to c = c // a

3.6 IDENTITY OPERATOR

Operator	Description	Example
is	Evaluates to true if the variables on either side of the operator point to the same object and false otherwise.	x is y, here **is** results in 1 if id(x) equals id(y).
is not	Evaluates to false if the variables on either side of the operator point to the same object and true otherwise.	x is not y, here **is not** results in 1 if id(x) is not equal to id(y

3.7 COMPARISON OPERATOR

Operator	Description	Example
& Binary AND	Operator copies a bit to the result if it exists in both operands	(a & b) (means 0000 1100)
| Binary OR	It copies a bit if it exists in either operand.	(a | b) = 61 (means 0011 1101)
^ Binary XOR	It copies the bit if it is set in one operand but not both.	(a ^ b) = 49 (means 0011 0001)
~ Binary Ones Complement	It is unary and has the effect of 'flipping' bits.	(~a) = -61 (means 1100 0011 in 2's complement form due to a signed binary number.
<< Binary Left Shift	The left operands value is moved left by the number of bits specified by the right operand.	a << 2 = 240 (means 1111 0000)
>> Binary Right Shift	The left operands value is moved right by the number of bits specified by the right operand.	a >> 2 = 15 (means 0000 1111)

3.8 LOGICAL OPERATOR

Operator	Description	Example
and Logical AND	If both the operands are true then condition becomes true.	(a and b) is true.
or Logical OR	If any of the two operands are non-zero then condition becomes true.	(a or b) is true.
not Logical NOT	Used to reverse the logical state of its operand.	Not(a and b) is false.

3.9 Membership Operators

Operator	Description	Example
in	Evaluates to true if it finds a variable in the specified sequence and false otherwise.	x in y, here in results in a 1 if x is a member of sequence y.
not in	Evaluates to true if it does not finds a variable in the specified sequence and false otherwise.	x not in y, here not in results in a 1 if x is not a member of sequence y.

Python Operators Precedence

Operator	Description
**	Exponentiation (raise to the power)
~ + -	Complement, unary plus and minus (method names for the last two are +@ and -@)

* / % //	Multiply, divide, modulo and floor division
+ -	Addition and subtraction
>> <<	Right and left bitwise shift
&	Bitwise 'AND'
^ |	Bitwise exclusive `OR' and regular `OR'
<= < > >=	Comparison operators
<> == !=	Equality operators
= %= /= //= -= += *= **=	Assignment operators
is is not	Identity operators
in not in	Membership operators
not or and	Logical operators

3.10 LIST

The list is a most versatile data type available in Python which can be written as a list of comma-separated values (items) between square brackets. Important thing about a list is that items in a list need not be of the same type.

Creating a list is as simple as putting different comma-separated values between square brackets. For example −

```
list1 = ['physics', 'chemistry', 1997, 2000];
list2 = [1, 2, 3, 4, 5 ];
list3 = ["a", "b", "c", "d"]
```

Basic List Operations

Lists respond to the + and * operators much like strings; they mean concatenation and repetition here too, except that the result is a new list, not a string.

Python Expression	Results	Description
len([1, 2, 3])	3	Length
[1, 2, 3] + [4, 5, 6]	[1, 2, 3, 4, 5, 6]	Concatenation
['Hi!'] * 4	['Hi!', 'Hi!', 'Hi!', 'Hi!']	Repetition
3 in [1, 2, 3]	True	Membership
for x in [1, 2, 3]: print x,	1 2 3	Iteration

Built-in List Functions & Methods:

Python includes the following list functions −

SN	Function with Description
1	cmp(list1, list2) Compares elements of both lists.
2	len(list) Gives the total length of the list.

3	max(list) Returns item from the list with max value.
4	min(list) Returns item from the list with min value.
5	list(seq) Converts a tuple into list.

Python includes following list methods

SN	Methods with Description
1	list.append(obj) Appends object obj to list
2	list.count(obj) Returns count of how many times obj occurs in list
3	list. extend(seq) Appends the contents of seq to list
4	list.index(obj) Returns the lowest index in list that obj appears
5	list.insert(index, obj) Inserts object obj into list at offset index
6	list.pop(obj=list[-1]) Removes and returns last object or obj from list
7	list.remove(obj)

	Removes object obj from list
8	list.reverse() Reverses objects of list in place
9	list.sort([func]) Sorts objects of list, use compare function if given

3.11 TUPLES

A tuple is a sequence of immutable Python objects. Tuples are sequences, just like lists. The differences between tuples and lists are, the tuples cannot be changed unlike lists and tuples use parentheses, whereas lists use square brackets.

Creating a tuple is as simple as putting different comma-separated values. Optionally we can put these comma-separated values between parentheses also. For example −

```
tup1 = ('physics', 'chemistry', 1997, 2000);

tup2 = (1, 2, 3, 4, 5 );

tup3 = "a", "b", "c", "d";
```

The empty tuple is written as two parentheses containing nothing −

```
tup1 = ();
```

To write a tuple containing a single value you have to include a comma, even though there is only one value –

```
tup1 = (50,);
```

Like string indices, tuple indices start at 0, and they can be sliced, concatenated, and so on.

- **Accessing Values in Tuples:**

To access values in tuple, use the square brackets for slicing along with the index or indices to obtain value available at that index. For example –

```
tup1 = ('physics', 'chemistry', 1997, 2000);

tup2 = (1, 2, 3, 4, 5, 6, 7 );

print "tup1[0]: ", tup1[0]
```

```
print "tup2[1:5]: ", tup2[1:5]
```

When the code is executed, it produces the following result −

```
tup1[0]:  physics
tup2[1:5]:  [2, 3, 4, 5]
```

Updating Tuples:

Tuples are immutable which means you cannot update or change the values of tuple elements. We are able to take portions of existing tuples to create new tuples as the following example demonstrates −

```
tup1 = (12, 34.56);
tup2 = ('abc', 'xyz');
tup3 = tup1 + tup2;
print tup3
```

When the above code is executed, it produces the following result −

```
(12, 34.56, 'abc', 'xyz')
```

Delete Tuple Elements

Removing individual tuple elements is not possible. There is, of course, nothing wrong with putting together another tuple with the undesired elements discarded.

To explicitly remove an entire tuple, just use the **del** statement. For example:

```
tup = ('physics', 'chemistry', 1997, 2000);
print tup
del tup;
print "After deleting tup : "
print tup
```

Basic Tuples Operations:

Python Expression	Results	Description
len((1, 2, 3))	3	Length
(1, 2, 3) + (4, 5, 6)	(1, 2, 3, 4, 5, 6)	Concatenation
('Hi!',) * 4	('Hi!', 'Hi!', 'Hi!', 'Hi!')	Repetition
3 in (1, 2, 3)	True	Membership
for x in (1, 2, 3): print x,	1 2 3	Iteration

Built-in Tuple Functions

SN	Function with Description
1	**cmp(tuple1, tuple2)**:Compares elements of both tuples.
2	**len(tuple)**:Gives the total length of the tuple.
3	**max(tuple)**:Returns item from the tuple with max value.
4	**min(tuple)**:Returns item from the tuple with min value.
5	**tuple(seq)**:Converts a list into tuple.

3.12 DICTIONARY

Each key is separated from its value by a colon (:), the items are separated by commas, and the whole thing is enclosed in curly braces. An empty dictionary without any items is written with just two curly braces, like this: {}.

Keys are unique within a dictionary while values may not be. The values of a dictionary can be of any type, but the keys must be of an immutable data type such as strings, numbers, or tuples.

Accessing Values in Dictionary:

To access dictionary elements, you can use the familiar square brackets along with the key to obtain its value. Following is a simple example −

```
dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}


print "dict['Name']: ", dict['Name']
print "dict['Age']: ", dict['Age']
```

Result –

```
dict['Name']:  Zara
dict['Age']:  7
```

Updating Dictionary

We can update a dictionary by adding a new entry or a key-value pair, modifying an existing entry, or deleting an existing entry as shown below in the simple example −

```
dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}


dict['Age'] = 8; # update existing entry
dict['School'] = "DPS School"; # Add new entry
print "dict['Age']: ", dict['Age']
print "dict['School']: ", dict['School']
```

Result −

```
dict['Age']:  8
dict['School']:  DPS School
```

Delete Dictionary Elements

We can either remove individual dictionary elements or clear the entire contents of a dictionary. You can also delete entire dictionary in a single operation.

To explicitly remove an entire dictionary, just use the **del** statement. Following is a simple example –

```
dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}


del dict['Name']; # remove entry with key 'Name'
dict.clear();     # remove all entries in dict
del dict ;        # delete entire dictionary


print "dict['Age']: ", dict['Age']
print "dict['School']: ", dict['School']
```

Built-in Dictionary Functions & Methods –

Python includes the following dictionary functions −

SN	Function with Description
1	cmp(dict1, dict2) Compares elements of both dict.
2	len(dict) Gives the total length of the dictionary. This would be equal to the number of items in the dictionary.

3	str(dict) Produces a printable string representation of a dictionary
4	type(variable) Returns the type of the passed variable. If passed variable is dictionary, then it would return a dictionary type.

Python includes following dictionary methods −

SN	Methods with Description
1	**dict.clear()**:Removes all elements of dictionary *dict*
2	**dict. Copy()**:Returns a shallow copy of dictionary *dict*
3	**dict.fromkeys()**:Create a new dictionary with keys from seq and values *set* to *value*.
4	**dict.get(key, default=None)**:For *key* key, returns value or default if key not in dictionary
5	**dict.has_key(key)**:Returns *true* if key in dictionary *dict*, *false* otherwise
6	**dict.items()**:Returns a list of *dict*'s (key, value) tuple pairs
7	**dict.keys()**:Returns list of dictionary dict's keys
8	**dict.setdefault(key, default=None)**:Similar to get(), but will set dict[key]=default if *key* is not already in dict
9	**dict.update(dict2)**:Adds dictionary *dict2*'s key-values pairs to *dict*
10	**dict.values()**:Returns list of dictionary *dict*'s values

A function is a block of organized, reusable code that is used to perform a single, related action. Functions provide better modularity for your application and a high degree of code reusing. Python gives you many built-in functions like print(), etc. but you can also create your own functions. These functions are called *user-defined functions.*

Defining a Function

Simple rules to define a function in Python.

- Function blocks begin with the keyword def followed by the function name and parentheses (( )).
- Any input parameters or arguments should be placed within these parentheses. You can also define parameters inside these parentheses.
- The first statement of a function can be an optional statement - the documentation string of the function or *docstring*.
- The code block within every function starts with a colon (:) and is indented.
- The statement return [expression] exits a function, optionally passing back an expression to the caller. A return statement with no arguments is the same as return None.

```
def functionname( parameters ):
   "function_docstring"
   function_suite
   return [expression]
```

Calling a Function

Defining a function only gives it a name, specifies the parameters that are to be included in the function and structures the blocks of code.Once the basic structure of a function is finalized, you can execute it by calling it from another function or directly from the Python prompt. Following is the example to call printme() function −

```
# Function definition is here
def printme( str ):
```

```
   "This prints a passed string into this function"
   print str
   return;
# Now you can call printme function
printme("I'm first call to user defined function!")
printme("Again second call to the same function")
```

When the above code is executed, it produces the following result −

```
I'm first call to user defined function!
Again second call to the same function
```

Function Arguments

You can call a function by using the following types of formal arguments:

- Required arguments
- Keyword arguments
- Default arguments
- Variable-length arguments

Scope of Variables

All variables in a program may not be accessible at all locations in that program. This depends on where you have declared a variable.

The scope of a variable determines the portion of the program where you can access a particular identifier. There are two basic scopes of variables in Python −

Global variables Local variables

Global vs. Local variables

Variables that are defined inside a function body have a local scope, and those defined outside have a global scope.

This means that local variables can be accessed only inside the function in which they are declared, whereas global variables can be accessed throughout the program body by all functions. When you call a function, the variables declared inside it are brought into scope. Following is a simple example −

```
total = 0; # This is global variable.
# Function definition is here
def sum( arg1, arg2 ):
   # Add both the parameters and return them."
   total = arg1 + arg2; # Here total is local variable.
   print "Inside the function local total : ", total
   return total;
sum( 10, 20 );
print "Outside the function global total : ", total
```

Result −

```
Inside the function local total :  30
Outside the function global total :  0
```

A module allows you to logically organize your Python code. Grouping related code into a module makes the code easier to understand and use. A module is a Python object with arbitrarily named attributes that you can bind and reference.Simply, a module is a file consisting of Python code. A module can define functions, classes and variables. A module can also include runnable code.

Example:

The Python code for a module named *aname* normally resides in a file named *aname.py*. Here's an example of a simple module, support.py

```
def print_func( par ):
   print "Hello : ", par
   return
```

The *import* Statement

The *import* has the following syntax:

```
import module1[, module2[,... moduleN]
```

When the interpreter encounters an import statement, it imports the module if the module is present in the search path. A search path is a list of directories that the interpreter searches before importing a module. For example, to import the module support.py, you need to put the following command at the top of the script −

A module is loaded only once, regardless of the number of times it is imported. This prevents the module execution from happening over and over again if multiple imports occur.

Packages in Python

A package is a hierarchical file directory structure that defines a single Python application environment that consists of modules and sub packages and sub-sub packages.

Consider a file *Pots.py* available in *Phone* directory. This file has following line of source code −

```
def Pots():
  print "I'm Pots Phone"
```

Similar way, we have another two files having different functions with the same name as above −

- *Phone/Isdn.py* file having function Isdn()
- *Phone/G3.py* file having function G3()

Now, create one more file __init__.py in *Phone* directory −

- Phone/__init__.py

To make all of your functions available when you've imported Phone,to put explicit import statements in __init__.py as follows −

```
from Pots import Pots
from Isdn import Isdn
```

```
from G3 import G3
```

After you add these lines to __init__.py, you have all of these classes available when you import the Phone package.

```
# Now import your Phone Package.
import Phone
Phone.Pots()
Phone.Isdn()
Phone.G3()
```

RESULT:

```
I'm Pots Phone
I'm 3G Phone
I'm ISDN Phone
```

In the above example, we have taken example of a single functions in each file, but you can keep multiple functions in your files. You can also define different Python classes in those files and then you can create your packages out of those classes.

This chapter covers all the basic I/O functions available in Python.

Printing to the Screen

The simplest way to produce output is using the *print* statement where you can pass zero or more expressions separated by commas. This function converts the expressions you pass into a string and writes the result to standard output as follows −

```
print "Python is really a great language,", "isn't it?"
```

Result:

```
Python is really a great language, isn't it?
```

Reading Keyboard Input

Python provides two built-in functions to read a line of text from standard input, which by default comes from the keyboard. These functions are −

- raw_input
- input

The *raw_input* Function

The *raw_input([prompt])* function reads one line from standard input and returns it as a string (removing the trailing newline).

```
str = raw_input("Enter your input: ");
print "Received input is : ", str
```

This prompts you to enter any string and it would display same string on the screen. When I typed "Hello Python!", its output is like this −

```
Enter your input: Hello Python
Received input is :  Hello Python
```

The *input* Function

The *input([prompt])* function is equivalent to raw_input, except that it assumes the input is a valid Python expression and returns the evaluated result to you.

```
str = input("Enter your input: ");
print "Received input is : ", str
```

This would produce the following result against the entered input −

```
Enter your input: [x*5 for x in range(2,10,2)]
Recieved input is :  [10, 20, 30, 40]
```

Opening and Closing Files

Until now, you have been reading and writing to the standard input and output. Now, we will see how to use actual data files.

Python provides basic functions and methods necessary to manipulate files by default. You can do most of the file manipulation using a **file** object.

The *open* Function

Before you can read or write a file, you have to open it using Python's built-in *open()* function. This function creates a **file** object, which would be utilized to call other support methods associated with it.

Syntax

```
file object = open(file_name [, access_mode][, buffering])
```

Modes	Description
r	Opens a file for reading only. The file pointer is placed at the beginning of the file. This is the default mode.
rb	Opens a file for reading only in binary format. The file pointer is placed at the beginning of the file. This is the default mode.
r+	Opens a file for both reading and writing. The file pointer placed at the beginning of the file.
rb+	Opens a file for both reading and writing in binary format. The file pointer placed at the beginning of the file.
w	Opens a file for writing only. Overwrites the file if the file exists. If the file does not exist, creates a new file for writing.
wb	Opens a file for writing only in binary format. Overwrites the file if the file exists. If the file does not exist, creates a new file for writing.
w+	Opens a file for both writing and reading. Overwrites the existing file if the file exists. If the file does not exist, creates a new file for reading and writing.
wb+	Opens a file for both writing and reading in binary format. Overwrites the existing file if the file exists. If the file does not exist, creates a new file for reading and writing.
a	Opens a file for appending. The file pointer is at the end of the file if the file exists. That is, the file is in the append mode. If the file does not exist, it creates a new file for writing.
ab	Opens a file for appending in binary format. The file pointer is at the end of the file if the file exists. That is, the file is in the append mode. If the file does not exist, it creates a new file for writing.
a+	Opens a file for both appending and reading. The file pointer is at the end of the file if the file exists. The file opens in the append mode. If the file does not exist, it creates a new file for reading and writing.

ab+	Opens a file for both appending and reading in binary format. The file pointer is at the end of the file if the file exists. The file opens in the append mode. If the file does not exist, it creates a new file for reading and writing.

Here are parameter details:

- **file_name:** The file_name argument is a string value that contains the name of the file that you want to access.
- **access_mode:** The access_mode determines the mode in which the file has to be opened, i.e., read, write, append, etc. A complete list of possible values is given below in the table. This is optional parameter and the default file access mode is read (r).
- **buffering:** If the buffering value is set to 0, no buffering takes place. If the buffering value is 1, line buffering is performed while accessing a file. If you specify the buffering value as an integer greater than 1, then buffering action is performed with the indicated buffer size. If negative, the buffer size is the system default(default behavior).

Here is a list of the different modes of opening a file −

The *file* Object Attributes

Attribute	Description
file.closed	Returns true if file is closed, false otherwise.
file.mode	Returns access mode with which file was opened.
file.name	Returns name of the file.
file.softspace	Returns false if space explicitly required with print, true otherwise.

Once a file is opened and you have one *file* object, you can get various information related to that file.

Here is a list of all attributes related to file object:

Example

```
# Open a file
fo = open("foo.txt", "wb")
print "Name of the file: ", fo.name
print "Closed or not : ", fo.closed
print "Opening mode : ", fo.mode
print "Softspace flag : ", fo.softspace
```

This produces the following result −

```
Name of the file:  foo.txt
Closed or not :  False
Opening mode :  wb
Softspace flag :  0
```

The *close()* Method

The close() method of a *file* object flushes any unwritten information and closes the file object, after which no more writing can be done.Python automatically closes a file when the reference object of a file is reassigned to another file. It is a good practice to use the close() method to close a file.

Syntax

```
fileObject.close();
```

Example

```
# Open a file
fo = open("foo.txt", "wb")
print "Name of the file: ", fo.name
# Close opend file
fo.close()
```

Result −

```
Name of the file:  foo.txt
```

Reading and Writing Files

The *file* object provides a set of access methods to make our lives easier. We would see how to use *read()* and *write()* methods to read and write files.

The *write()* Method

The *write()* method writes any string to an open file. It is important to note that Python strings can have binary data and not just text.The write() method does not add a newline character ('\n') to the end of the string **Syntax**

```
fileObject.write(string);
```

Here, passed parameter is the content to be written into the opened file. **Example**

```
# Open a file
fo = open("foo.txt", "wb")
fo.write( "Python is a great language.\nYeah its great!!\n");

# Close opend file
fo.close()
```

The above method would create *foo.txt* file and would write given content in that file and finally it would close that file. If you would open this file, it would have following content.

```
Python is a great language.
Yeah its great!!
```

The *read()* Method

The *read()* method reads a string from an open file. It is important to note that Python strings can have binary data. apart from text data.

Syntax

```
fileObject.read([count]);
```

Here, passed parameter is the number of bytes to be read from the opened file. This method starts reading from the beginning of the file and if *count* is missing, then it tries to read as much as possible, maybe until the end of file.

Example

Let's take a file *foo.txt*, which we created above.

```
# Open a file
fo = open("foo.txt", "r+")
str = fo.read(10);
print "Read String is : ", str
# Close opend file
fo.close()
```

This produces the following result −

```
Read String is :  Python is
```

File Positions

The *tell()* method tells you the current position within the file; in other words, the next read or write will occur at that many bytes from the beginning of the file.

The *seek(offset[, from])* method changes the current file position. The *offset* argument indicates the number of bytes to be moved. The *from* argument specifies the reference position from where the bytes are to be moved.

If *from* is set to 0, it means use the beginning of the file as the reference position and 1 means use the current position as the reference position and if it is set to 2 then the end of the file would be taken as the reference position.

Example

Let us take a file *foo.txt*, which we created above.

```
# Open a file
fo = open("foo.txt", "r+")
str = fo.read(10);
print "Read String is : ", str

# Check current position
position = fo.tell();
print "Current file position : ", position

# Reposition pointer at the beginning once again
position = fo.seek(0, 0);
str = fo.read(10);
print "Again read String is : ", str
# Close opend file
fo.close()
```

This produces the following result −

```
Read String is :  Python is
Current file position :  10
Again read String is :  Python is
```

Renaming and Deleting Files

Python **os** module provides methods that help you perform file-processing operations, such as renaming and deleting files.

To use this module you need to import it first and then you can call any related functions.

The rename() Method

The *rename()* method takes two arguments, the current filename and the new filename.

Syntax

```
os.rename(current_file_name, new_file_name)
```

Example

Following is the example to rename an existing file *test1.txt*:

```
import os

# Rename a file from test1.txt to test2.txt
os.rename( "test1.txt", "test2.txt" )
```

The *remove()* Method

You can use the *remove()* method to delete files by supplying the name of the file to be deleted as the argument.

Syntax

```
os.remove(file_name)
```

Example

Following is the example to delete an existing file *test2.txt* −

```
#!/usr/bin/python
import os

# Delete file test2.txt
os.remove("text2.txt")
```

Directories in Python

All files are contained within various directories, and Python has no problem handling these too. The **os** module has several methods that help you create, remove, and change directories.

The *mkdir()* Method

You can use the *mkdir()* method of the **os** module to create directories in the current directory. You need to supply an argument to this method which contains the name of the directory to be created.

Syntax

```
os.mkdir("newdir")
```

Example

Following is the example to create a directory *test* in the current directory −

```
#!/usr/bin/python
import os

# Create a directory "test"
os.mkdir("test")
```

The *chdir()* Method

You can use the *chdir()* method to change the current directory. The chdir() method takes an argument, which is the name of the directory that you want to make the current directory.

Syntax

```
os.chdir("newdir")
```

Example

Following is the example to go into "/home/newdir" directory −

```
#!/usr/bin/python
import os

# Changing a directory to "/home/newdir"
os.chdir("/home/newdir")
```

The *getcwd()* Method

The *getcwd()* method displays the current working directory.

Syntax

```
os.getcwd()
```

Example

Following is the example to give current directory −

```
import os
```

EXCEPTION NAME	DESCRIPTION
Exception	Base class for all exceptions
StopIteration	Raised when the next() method of an iterator does not point to any object.
SystemExit	Raised by the sys.exit() function.
StandardError	Base class for all built-in exceptions except StopIteration and SystemExit.
ArithmeticError	Base class for all errors that occur for numeric calculation.
OverflowError	Raised when a calculation exceeds maximum limit for a numeric type.
FloatingPointError	Raised when a floating point calculation fails.
ZeroDivisionError	Raised when division or modulo by zero takes place for all numeric types.
AssertionError	Raised in case of failure of the Assert statement.
AttributeError	Raised in case of failure of attribute reference or assignment.
EOFError	Raised when there is no input from either the raw_input() or input() function and the end of file is reached.
ImportError	Raised when an import statement fails.
KeyboardInterrupt	Raised when the user interrupts program execution, usually by pressing Ctrl+c.

LookupError	Base class for all lookup errors.
IndexError KeyError	Raised when an index is not found in a sequence. Raised when the specified key is not found in the dictionary.
NameError	Raised when an identifier is not found in the local or global namespace.
UnboundLocalError EnvironmentError	Raised when trying to access a local variable in a function or method but no value has been assigned to it. Base class for all exceptions that occur outside the Python environment.
IOError IOError	Raised when an input/ output operation fails, such as the print statement or the open() function when trying to open a file that does not exist. Raised for operating system-related errors.
SyntaxError IndentationError	Raised when there is an error in Python syntax. Raised when indentation is not specified properly.
SystemError	Raised when the interpreter finds an internal problem, but when this error is encountered the Python interpreter does not exit.
SystemExit	Raised when Python interpreter is quit by using the sys.exit() function. If not handled in the code, causes the interpreter to exit.
TypeError	Raised when an operation or function is attempted that is invalid for the specified data type.

ValueError	Raised when the built-in function for a data type has the valid type of arguments, but the arguments have invalid values specified.
RuntimeError	Raised when a generated error does not fall into any category.
NotImplementedError	Raised when an abstract method that needs to be implemented in an inherited class is not actually implemented.

```
os.getcwd()
```

The *rmdir()* Method

The *rmdir()* method deletes the directory, which is passed as an argument in the method.

Before removing a directory, all the contents in it should be removed.

Syntax:

```
os.rmdir('dirname')
```

Example

Following is the example to remove "/tmp/test" directory. It is required to give fully qualified name of the directory, otherwise it would search for that directory in the current directory.

```
import os
# This would  remove "/tmp/test"  directory.
os.rmdir( "/tmp/test"  )
```

File & Directory Related Methods

There are three important sources, which provide a wide range of utility methods to handle and manipulate files & directories on Windows and Unix operating systems. They are as follows −

- File Object Methods: The *file* object provides functions to manipulate files.
- OS Object Methods: This provides methods to process files as well as directories.

Python provides two very important features to handle any unexpected error in your Python programs and to add debugging capabilities in them −

- **Exception Handling:** This would be covered in this tutorial. Here is a list standard Exceptions available in Python: Standard Exceptions.

- **Assertions:** This would be covered in Assertions in Python

List of Standard Exceptions −

What is Exception?

An exception is an event, which occurs during the execution of a program that disrupts the normal flow of the program's instructions. In general, when a Python script encounters a situation that it cannot cope with, it raises an exception. An exception is a Python object that represents an error.

When a Python script raises an exception, it must either handle the exception immediately otherwise it terminates and quits.

Handling an exception

If you have some *suspicious* code that may raise an exception, you can defend your program by placing the suspicious code in a **try:** block. After the try: block, include an **except:** statement, followed by a block of code which handles the problem as elegantly as possible.

The Python standard for database interfaces is the Python DB-API. Most Python database interfaces adhere to this standard.

You can choose the right database for your application. Python Database API supports a wide range of database servers such as −

- GadFly
- mSQL
- MySQL
- PostgreSQL
- Microsoft SQL Server 2000
- Informix
- Interbase
- Oracle

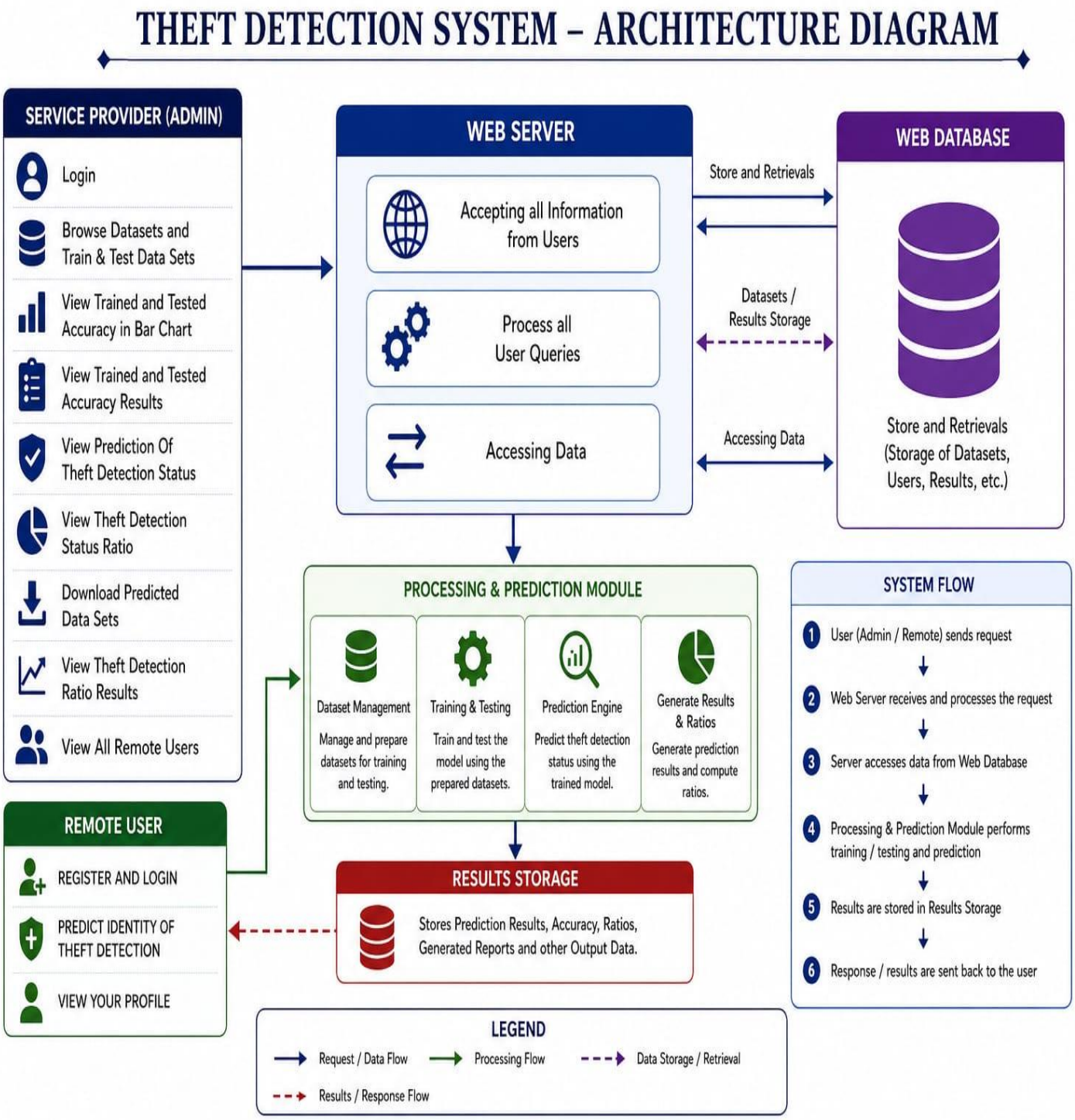
- Sybase

The DB API provides a minimal standard for working with databases using Python structures and syntax wherever possible. This API includes the following:

- Importing the API module.
- Acquiring a connection with the database.
- Issuing SQL statements and stored procedures.

4. SYSTEM ARCHITECTURE

4.1 SYSTEM OVERVIEW



4.2 Modules

Service Provider

In this module, the Service Provider has to login by using valid user name and password. After login successful he can do some operations such as

Login, Browse Datasets and Train & Test Data Sets, View Trained and Tested Accuracy in Bar Chart, View Trained and Tested Accuracy Results, View Prediction Of Theft Detection Status, View Theft Detection Status Ratio, Download Predicted Data Sets, View Theft Detection Ratio Results, View All Remote Users.

View and Authorize Users

In this module, the admin can view the list of users who all registered. In this, the admin can view the user's details such as, user name, email, address and admin authorizes the users.

Remote User

In this module, there are n numbers of users are present. User should register before doing any operations. Once user registers, their details will be stored to the database. After registration successful, he has to login by using authorized user name and password. Once Login is successful user will do some operations like REGISTER AND LOGIN, PREDICT IDENTITY OF THEFT DETECTION,, VIEW YOUR PROFILE.

Decision tree classifiers

Decision tree classifiers are used successfully in many diverse areas. Their most important feature is the capability of capturing descriptive decision making knowledge from the supplied data. Decision tree can be generated from training sets. The procedure for such generation based on the set of objects (S), each belonging to one of the classes C1, C2, …, Ck is as follows:

Step 1. If all the objects in S belong to the same class, for example Ci, the decision tree for S consists of a leaf labeled with this class

Step 2. Otherwise, let T be some test with possible outcomes O1, O2,…, On. Each object in S has one outcome for T so the test partitions S into subsets S1, S2,… Sn where each object in Si has outcome Oi for T. T becomes the root of the decision tree and for each outcome Oi we build a subsidiary decision tree by invoking the same procedure recursively on the set Si.

Gradient boosting

Gradient boosting is a machine learning technique used in regression and classification tasks, among others. It gives a prediction model in the form of an ensemble of weak prediction models, which are

typically decision trees.[1][2] When a decision tree is the weak learner, the resulting algorithm is called gradient-boosted trees; it usually outperforms random forest.A gradient-boosted trees model is built in a stage-wise fashion as in other boosting methods, but it generalizes the other methods by allowing optimization of an arbitrary differentiable loss function.

K-Nearest Neighbors (KNN)

- **Simple, but a very powerful classification algorithm**

- **Classifies based on a similarity measure**
- **Non-parametric**
- **Lazy learning**
- **Does not "learn" until the test example is given**

- **Whenever we have a new data to classify, we find its K-nearest neighbors from the training data**

Example

- **Training dataset consists of k-closest examples in feature space**
- **Feature space means, space with categorization variables (non-metric variables)**
- **Learning based on instances, and thus also works lazily because instance close to the input vector for test or prediction may take time to occur in the training dataset**

Logistic regression Classifiers

Logistic regression analysis studies the association between a categorical dependent variable and a set of independent (explanatory) variables. The name *logistic regression* is used when the dependent variable has only two values, such as 0 and 1 or Yes and No. The name *multinomial logistic regression* is usually reserved for the case when the dependent variable has three or more unique values, such as Married, Single, Divorced, or Widowed. Although the type of data used for the dependent variable is different from that of multiple regression, the practical use of the procedure is similar.

Logistic regression competes with discriminant analysis as a method for analyzing categorical-response variables. Many statisticians feel that logistic regression is more versatile and better suited for modeling most situations than is discriminant analysis. This is because logistic regression does not assume that the independent variables are normally distributed, as discriminant analysis does.

This program computes binary logistic regression and multinomial logistic regression on both numeric and categorical independent variables. It reports on the regression equation as well as the goodness of fit, odds ratios, confidence limits, likelihood, and deviance. It performs a comprehensive residual analysis including

diagnostic residual reports and plots. It can perform an independent variable subset selection search, looking for the best regression model with the fewest independent variables. It provides confidence intervals on predicted values and provides ROC curves to help determine the best cutoff point for classification. It allows you to validate your results by automatically classifying rows that are not used during the analysis.

Naïve Bayes

The naive bayes approach is a supervised learning method which is based on a simplistic hypothesis: it assumes that the presence (or absence) of a particular feature of a class is unrelated to the presence (or absence) of any other feature .
Yet, despite this, it appears robust and efficient. Its performance is comparable to other supervised learning techniques. Various reasons have been advanced in the literature. In this tutorial, we highlight an explanation based on the representation bias. The naive bayes classifier is a linear classifier, as well as linear discriminant analysis, logistic regression or linear SVM (support vector machine). The difference lies on the method of estimating the parameters of the classifier (the learning bias).

While the Naive Bayes classifier is widely used in the research world, it is not widespread among practitioners which want to obtain usable results. On the one hand, the researchers found especially it is very easy to program and implement it, its parameters are easy to estimate, learning is very fast even on very large databases, its accuracy is reasonably good in comparison to the other approaches. On the other hand, the final users do not obtain a model easy to interpret and deploy, they does not understand the interest of such a technique.

Thus, we introduce in a new presentation of the results of the learning process. The classifier is easier to understand, and its deployment is also made easier. In the first part of this tutorial, we present some theoretical aspects of the naive bayes classifier. Then, we implement the approach on a dataset with Tanagra. We compare the obtained results (the parameters of the model) to those obtained with other linear approaches such as the logistic regression, the linear discriminant analysis and the linear SVM. We note that the results are highly consistent. This largely explains the good performance of the method in comparison to others. In the second part, we use various tools on the same dataset (**Weka 3.6.0**, **R 2.9.2**, **Knime 2.1.1**, **Orange 2.0b** and **RapidMiner 4.6.0)**. We try above all to understand the obtained results.

Random Forest

Random forests or random decision forests are an ensemble learning method for classification, regression and other tasks that operates by constructing a multitude of decision trees at training time. For classification tasks, the output of the random forest is the class selected by most trees. For regression tasks, the mean or

average prediction of the individual trees is returned. Random decision forests correct for decision trees' habit of overfitting to their training set. Random forests generally outperform decision trees, but their accuracy is lower than gradient boosted trees. However, data characteristics can affect their performance.

The first algorithm for random decision forests was created in 1995 by Tin Kam Ho[1] using the random subspace method, which, in Ho's formulation, is a way to implement the "stochastic discrimination" approach to classification proposed by Eugene Kleinberg.

An extension of the algorithm was developed by Leo Breiman and Adele Cutler, who registered "Random Forests" as a trademark in 2006 (as of 2019, owned by Minitab, Inc.).The extension combines Breiman's "bagging" idea and random selection of features, introduced first by Ho[1] and later independently by Amit and Geman[13] in order to construct a collection of decision trees with controlled variance.

Random forests are frequently used as "blackbox" models in businesses, as they generate reasonable predictions across a wide range of data while requiring little configuration.

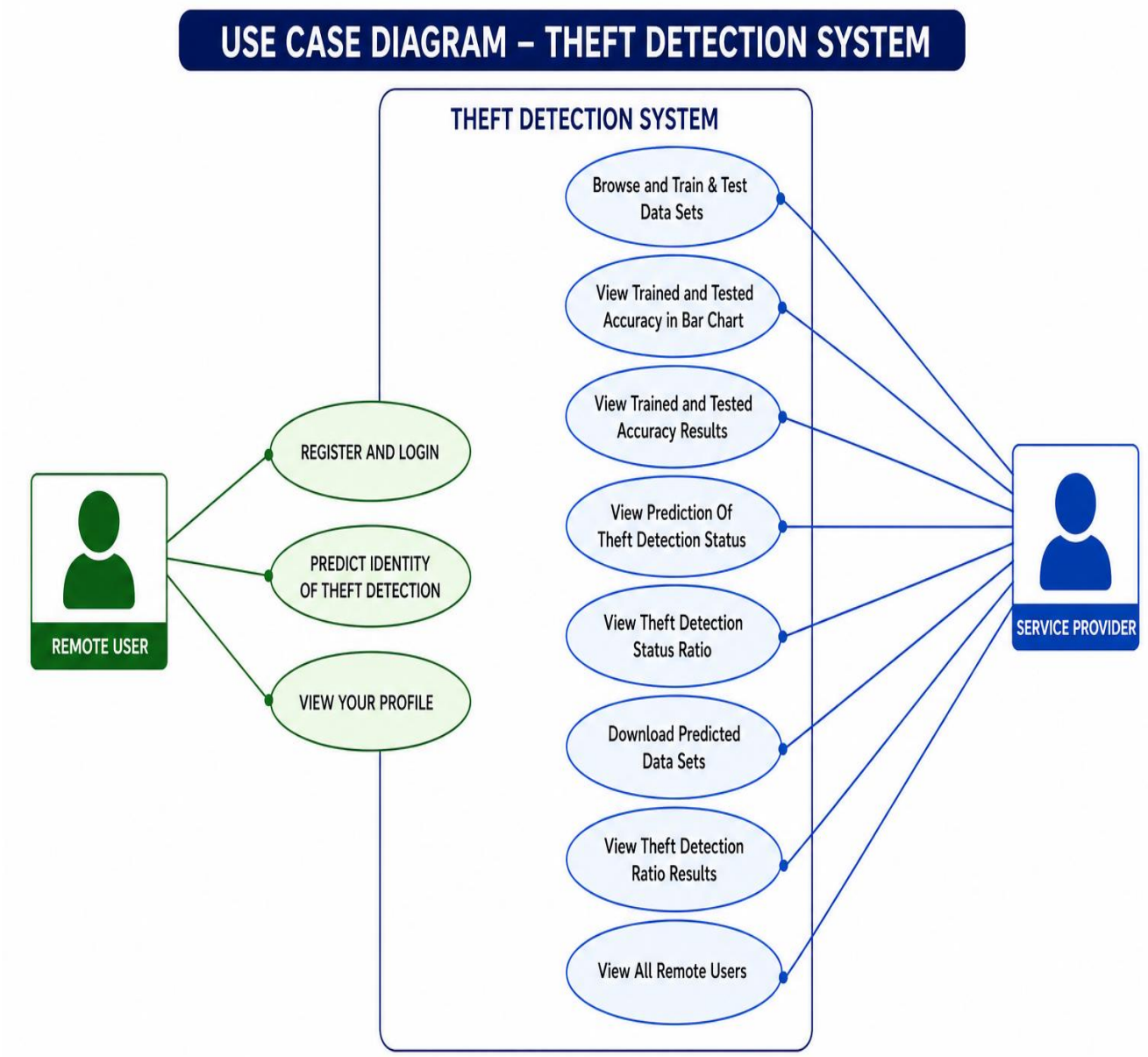
SVM

In classification tasks a discriminant machine learning technique aims at finding, based on an *independent and identically distributed* (*iid*) training dataset, a discriminant function that can correctly predict labels for newly acquired instances. Unlike generative machine learning approaches, which require computations of conditional probability distributions, a discriminant classification function takes a data point *x* and assigns it to one of the different classes that are a part of the classification task. Less powerful than generative approaches, which are mostly used when prediction involves outlier detection, discriminant approaches require fewer computational resources and less training data, especially for a multidimensional feature space and when only posterior probabilities are needed. From a geometric perspective, learning a classifier is equivalent to finding the equation for a multidimensional surface that best separates the different classes in the feature space.

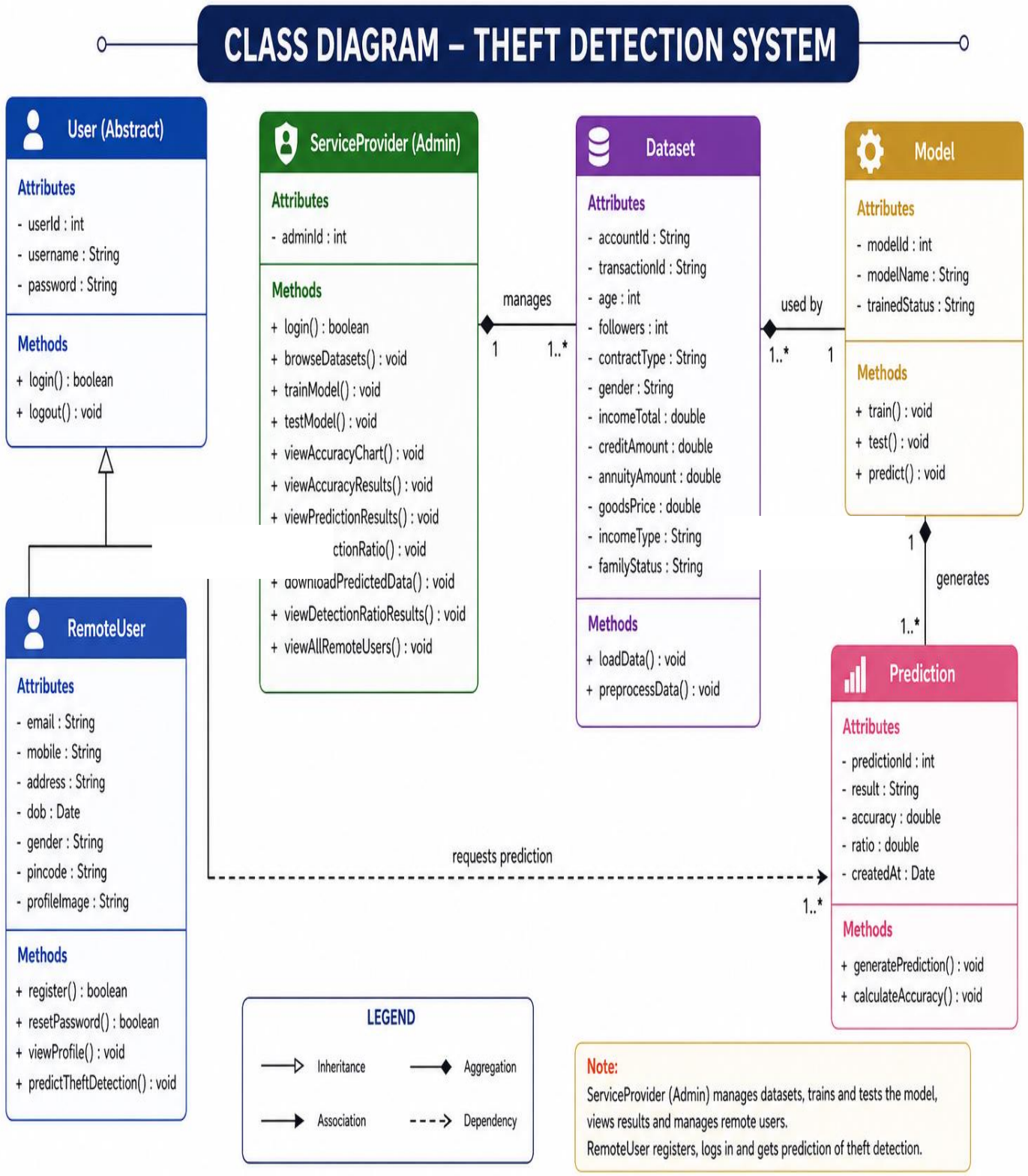
SVM is a discriminant technique, and, because it solves the convex optimization problem analytically, it always returns the same optimal hyperplane parameter—in contrast to *genetic algorithms* (*GAs*) or *perceptrons*, both of which are widely used for classification in machine learning. For perceptrons, solutions are highly dependent on the initialization and termination criteria. For a specific kernel that transforms the data from the input space to the feature space, training returns uniquely defined SVM model parameters for a given training set, whereas the perceptron and GA classifier models are different each time training is initialized. The aim of GAs and perceptrons is only to minimize error during training, which will translate into several hyperplanes' meeting this requirement.

5. SYSTEM DESIGN

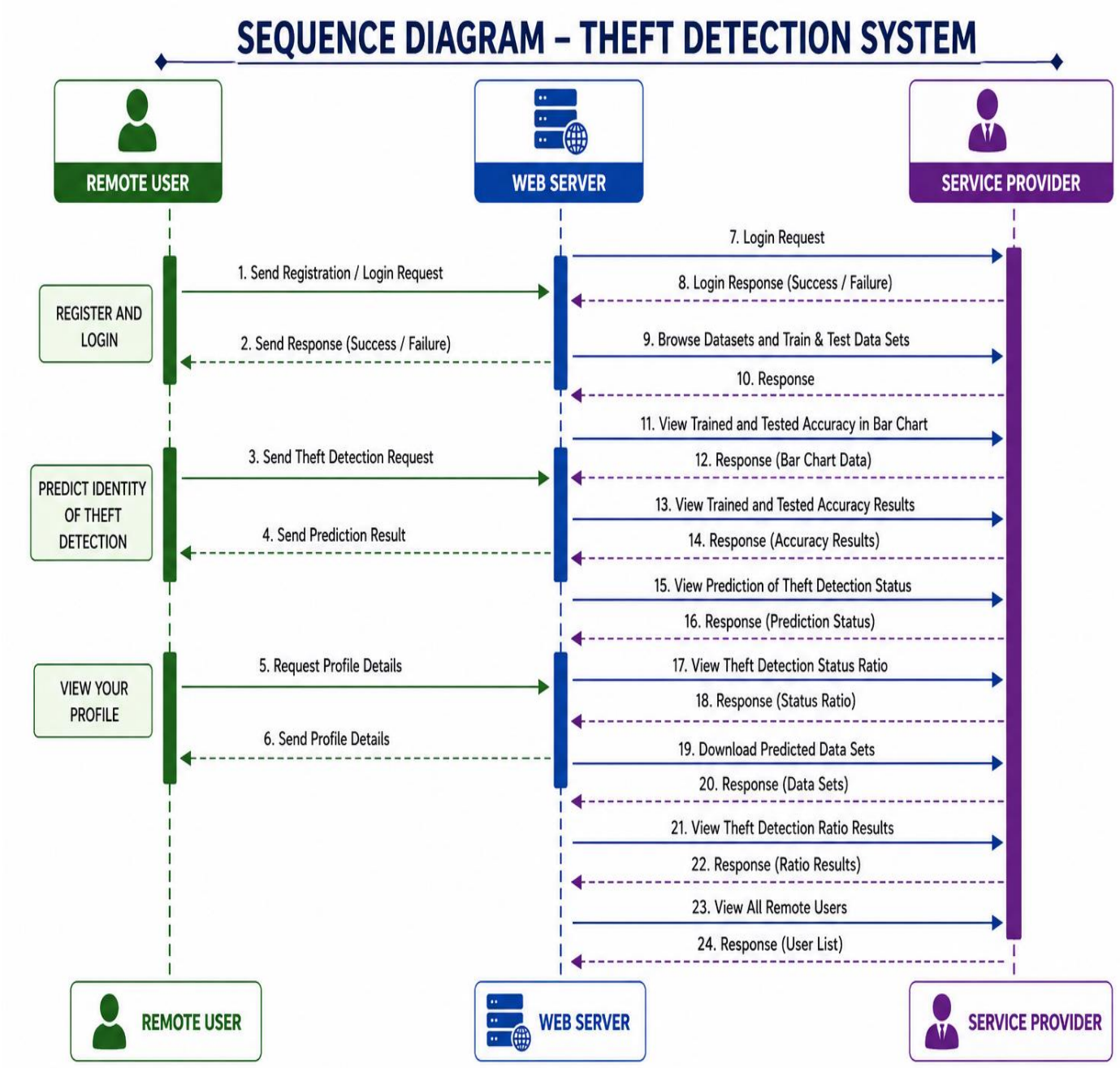
5.1 Use Case Diagram



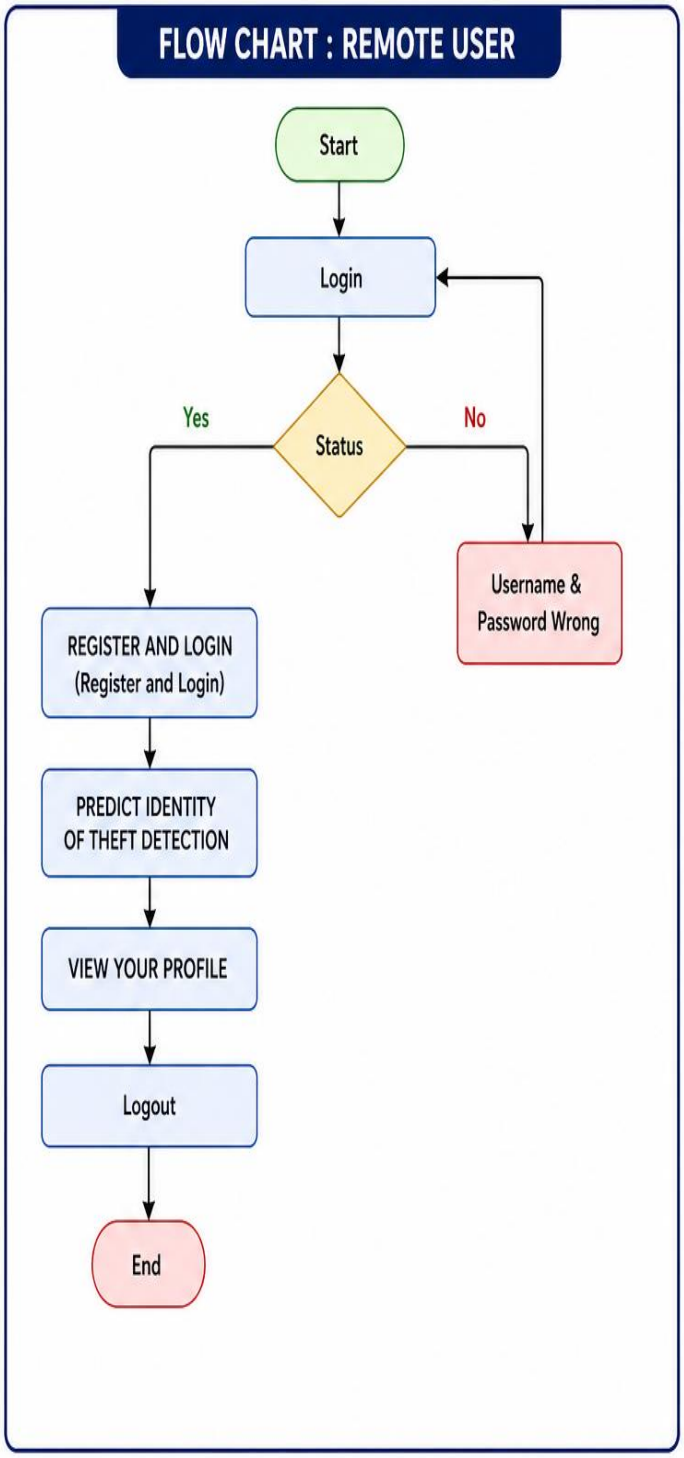
5.2Class Diagram :

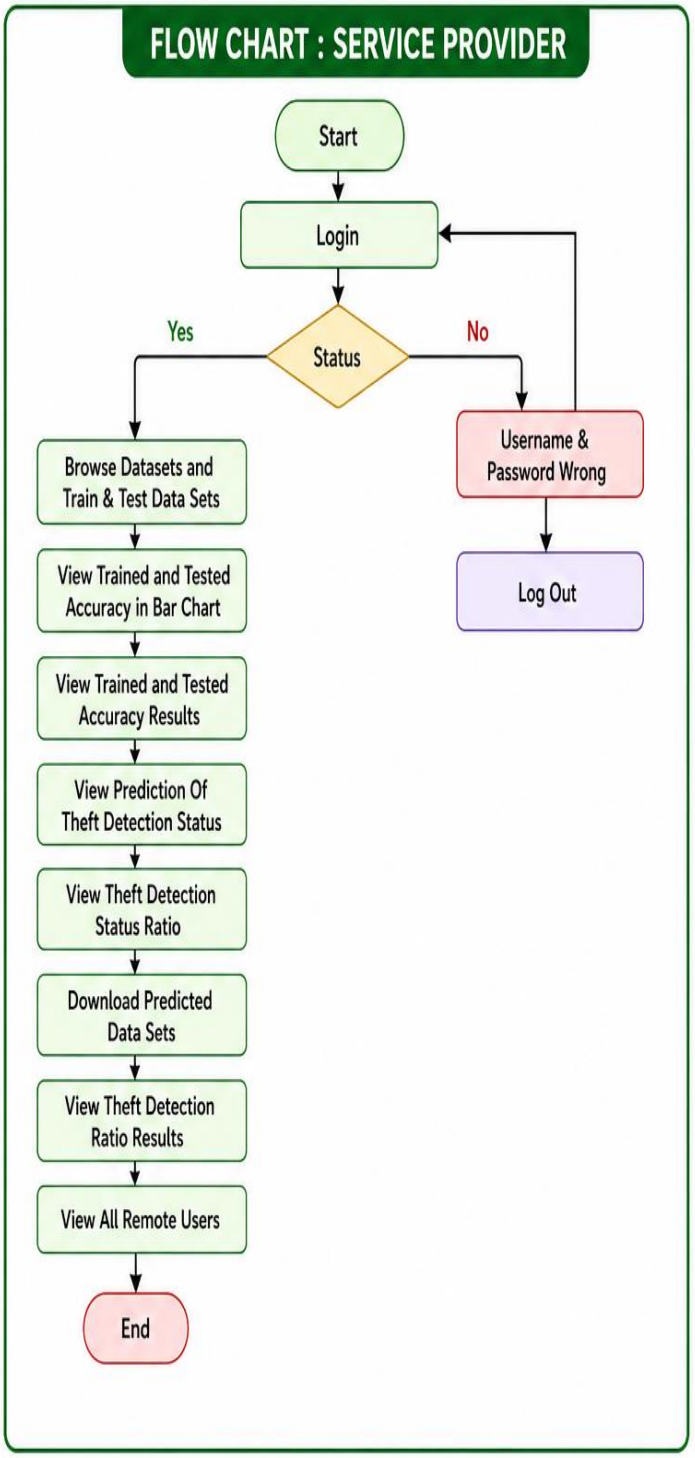


5.3 Sequence Diagram



5.4 Flow Chart





6. SYSTEM TESTING

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub assemblies, assemblies and/or a finished product It is the process of exercising software with the intent of ensuring that the

Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

TYPES OF TESTS

Unit testing

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application .it is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

Integration testing

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfaction, as shown by successfully unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

Functional test

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

Valid Input : identified classes of valid input must be accepted.

Invalid Input : identified classes of invalid input must be rejected.

Functions : identified functions must be exercised.

Output : identified classes of application outputs must be exercised.

Systems/Procedures: interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

System Test

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

White Box Testing

White Box Testing is a testing in which in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is purpose. It is used to test areas that cannot be reached from a black box level.

Black Box Testing

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box .you cannot “see” into it. The test provides inputs and responds to outputs without considering how the software works.

6.1 Unit Testing:

Unit testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases.

Test strategy and approach

Field testing will be performed manually and functional tests will be written in detail.

Test objectives

- All field entries must work properly.
- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed.

Features to be tested

- Verify that the entries are of the correct format
- No duplicate entries should be allowed
- All links should take the user to the correct page.

6.2 Integration Testing

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects.

The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

6.3 Acceptance Testing

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

SYSTEM TESTING

TESTING METHODOLOGIES

The following are the Testing Methodologies:

- **Unit Testing.**
- **Integration Testing.**
- **User Acceptance Testing.**
- **Output Testing.**
- **Validation Testing.**

Unit Testing

Unit testing focuses verification effort on the smallest unit of Software design that is the module. Unit testing exercises specific paths in a module's control structure to ensure complete coverage and maximum error detection. This test focuses on each module individually, ensuring that it functions properly as a unit. Hence, the naming is Unit Testing.

During this testing, each module is tested individually and the module interfaces are verified for the consistency with design specification. All important processing path are tested for the expected results. All error handling paths are also tested.

Integration Testing

Integration testing addresses the issues associated with the dual problems of verification and program construction. After the software has been integrated a set of high order tests are conducted. The main objective in this testing process is to take unit tested modules and builds a program structure that has been dictated by design.

The following are the types of Integration Testing:

1)Top Down Integration

This method is an incremental approach to the construction of program structure. Modules are integrated by moving downward through the control hierarchy, beginning with the main program module. The module subordinates to the main program module are incorporated into the structure in either a depth first or breadth first manner.

In this method, the software is tested from main module and individual stubs are replaced when the test proceeds downwards.

2. Bottom-up Integration

This method begins the construction and testing with the modules at the lowest level in the program structure. Since the modules are integrated from the bottom up, processing required for modules subordinate to a given level is always available and the need for stubs is eliminated. The bottom up integration strategy may be implemented with the following steps:

- The low-level modules are combined into clusters into clusters that perform a specific Software sub-function.
- A driver (i.e.) the control program for testing is written to coordinate test case input and output.
- The cluster is tested.
- Drivers are removed and clusters are combined moving upward in the program structure

The bottom up approaches tests each module individually and then each module is module is integrated with a main module and tested for functionality.

OTHER TESTING METHODOLOGIES

User Acceptance Testing

User Acceptance of a system is the key factor for the success of any system. The system under consideration is tested for user acceptance by constantly keeping in touch with the prospective system users at the time of developing and making changes wherever required. The system developed provides a friendly user interface that can easily be understood even by a person who is new to the system.

Output Testing

After performing the validation testing, the next step is output testing of the proposed system, since no system could be useful if it does not produce the required output in the specified format. Asking the users about the format required by them tests the outputs generated or displayed by the system under consideration. Hence the output format is considered in 2 ways – one is on screen and another in printed format.

Validation Checking

Validation checks are performed on the following fields.

Text Field:

The text field can contain only the number of characters lesser than or equal to its size. The text fields are alphanumeric in some tables and alphabetic in other tables. Incorrect entry always flashes and error message.

Numeric Field:

The numeric field can contain only numbers from 0 to 9. An entry of any character flashes an error messages. The individual modules are checked for accuracy and what it has to perform. Each module is subjected to test run along with sample data. The individually tested modules are integrated into a single system. Testing involves executing the real data information is used in the program the existence of any program defect is inferred from the output. The testing should be planned so that all the requirements are individually tested.

A successful test is one that gives out the defects for the inappropriate data and produces and output revealing the errors in the system.

Preparation of Test Data

Taking various kinds of test data does the above testing. Preparation of test data plays a vital role in the system testing. After preparing the test data the system under study is tested using that test data. While testing the system by using test data errors are again uncovered and corrected by using above testing steps and corrections are also noted for future use.

Using Live Test Data:

Live test data are those that are actually extracted from organization files. After a system is partially constructed, programmers or analysts often ask users to key in a set of data from their normal activities. Then, the systems person uses this data as a way to partially test the system. In other instances, programmers or analysts extract a set of live data from the files and have them entered themselves.

It is difficult to obtain live data in sufficient amounts to conduct extensive testing. And, although it is realistic data that will show how the system will perform for the typical processing requirement, assuming that the live data entered are in fact typical, such data generally will not test all combinations or formats that can enter the system. This bias toward typical values then does not provide a true systems test and in fact ignores the cases most likely to cause system failure.

Using Artificial Test Data:

Artificial test data are created solely for test purposes, since they can be generated to test all combinations of formats and values. In other words, the artificial data, which can quickly be prepared by a data generating utility program in the information systems department, make possible the testing of all login and control paths through the program.

The most effective test programs use artificial test data generated by persons other than those who wrote the programs. Often, an independent team of testers formulates a testing plan, using the systems specifications.

The package “Virtual Private Network” has satisfied all the requirements specified as per software requirement specification and was accepted.

USER TRAINING

Whenever a new system is developed, user training is required to educate them about the working of the

system so that it can be put to efficient use by those for whom the system has been primarily designed. For this purpose the normal working of the project was demonstrated to the prospective users. Its working is easily understandable and since the expected users are people who have good knowledge of computers, the use of this system is very easy.

MAINTAINENCE

This covers a wide range of activities including correcting code and design errors. To reduce the need for maintenance in the long run, we have more accurately defined the user's requirements during the process of system development. Depending on the requirements, this system has been developed to satisfy the needs to the largest possible extent. With development in technology, it may be possible to add many more features based on the requirements in future. The coding and designing is simple and easy to understand which will make maintenance easier.

TESTING STRATEGY :

A strategy for system testing integrates system test cases and design techniques into a well planned series of steps that results in the successful construction of software. The testing strategy must co-operate test planning, test case design, test execution, and the resultant data collection and evaluation .A strategy for software testing must accommodate low-level tests that are necessary to verify that a small source code segment has been correctly implemented as well as high level tests that validate major system functions against user requirements.

Software testing is a critical element of software quality assurance and represents the ultimate review of specification design and coding. Testing represents an interesting anomaly for the software. Thus, a series of testing are performed for the proposed system before the system is ready for user acceptance testing.

SYSTEM TESTING:

Software once validated must be combined with other system elements (e.g. Hardware, people, database). System testing verifies that all the elements are proper and that overall system function performance is achieved. It also tests to find discrepancies between the system and its original objective, current specifications and system documentation.

UNIT TESTING:

In unit testing different are modules are tested against the specifications produced during the design for the modules. Unit testing is essential for verification of the code produced during the coding phase, and hence the goals to test the internal logic of the modules. Using the detailed design description as a guide, important Conrail paths are tested to uncover errors within the boundary of the modules. This testing is carried out during the programming stage itself. In this type of testing step, each module was found to be

working satisfactorily as regards to the expected output from the module.

In Due Course, latest technology advancements will be taken into consideration. As part of technical build-up many components of the networking system will be generic in nature so that future projects can either use or interact with this. The future holds a lot to offer to the development and refinement of this project.

7. CONCLUSION

We investigate the feasibility of building a ladder from low-quality behavioral data to a high-performance behavioral model for user identification in OSNs. By deeply exploiting the complementary effect among OSN users' multidimensional behaviors, we propose a joint probabilistic generative model by integrating online and offline behaviors. When the designed joint model is applied to identity theft detection in OSNs, its comprehensive performance, in terms of the detection efficacy, response latency, and robustness, is validated by extensive evaluations on real-life OSN datasets. Particularly, the joint model significantly outperforms the existing fused model.

Our behavior-based method mainly aims at detecting identity thieves after the access control of the account is broken. Then, it is easy and promising to incorporate our method into traditional methods to solve the identity theft problem better.

8. REFERENCES

[1] J. Onaolapo, E. Mariconti, and G. Stringhini, "What happens after you

are pwnd: Understanding the use of leaked Webmail credentials in the wild," in *Proc. Internet Meas. Conf.*, Nov. 2016, pp. 65–79.
[2] A. Mohan, "A medical domain collaborative anomaly detection framework for identifying medical identity theft," in *Proc. Int. Conf. Collaboration Technol. Syst. (CTS)*, May 2014, pp. 428–435.
[3] Y.-A. de Montjoye, L. Radaelli, V. K. Singh, and A. S. Pentland, "Unique in the shopping mall: On the reidentifiability of credit card metadata," *Science*, vol. 347, no. 6221, pp. 536–539, Jan. 2015.
[4] P. Hyman, "Cybercrime: It's serious, but exactly how serious?" *Commun. ACM*, vol. 56, no. 3, pp. 18–20, Mar. 2013.
[5] L. Bilge, T. Strufe, D. Balzarotti, and E. Kirda, "All your contacts are belong to us: Automated identity theft attacks on social networks," in *Proc. 18th Int. Conf. World Wide Web (WWW)*, 2009, pp. 551–560.
[6] J. Lynch, "Identity theft in cyberspace: Crime control methods and their effectiveness in combating phishing attacks," *Berkeley Technol. Law J.*, vol. 20, no. 1, pp. 259–300, 2005.
[7] M. Egele, G. Stringhini, C. Kruegel, and G. Vigna, "Towards detecting compromised accounts on social networks," *IEEE Trans. Dependable Secure Comput.*, vol. 14, no. 4, pp. 447–460, Jul. 2017.
[8] T. C. Pratt, K. Holtfreter, and M. D. Reisig, "Routine online activity and Internet fraud targeting: Extending the generality of routine activity theory," *J. Res. Crime Delinquency*, vol. 47, no. 3, pp. 267–296, Aug. 2010.
[9] K. Thomas, C. Grier, J. Ma, V. Paxson, and D. Song, "Design and evaluation of a real-time URL spam filtering service," in *Proc. IEEE Symp. Secur. Privacy*, May 2011, pp. 447–462.
[10] H. Li *et al.*, "Bimodal distribution and co-bursting in review spam detection," in *Proc. 26th Int. Conf. World Wide Web*, Apr. 2017, pp. 1063–1072.
[11] A. M. Marshall and B. C. Tompsett, "Identity theft in an online world," *Comput. Law Secur. Rev.*, vol. 21, no. 2, pp. 128–137, Jan. 2005.
[12] B. Schneier, "Two-factor authentication: Too little, too late," *Commun. ACM*, vol. 48, no. 4, p. 136, Apr. 2005.
[13] M. V. Ruiz-Blondet, Z. Jin, and S. Laszlo, "CEREBRE: A novel method for very high accuracy event-related potential biometric identification,"

IEEE Trans. Inf. Forensics Security, vol. 11, no. 7, pp. 1618–1629, Jul. 2016.

[14] R. D. Labati, A. Genovese, E. Muñoz, V. Piuri, F. Scotti, and G. Sforza, "Biometric recognition in automated border control: A survey," *ACM Comput. Surv.*, vol. 49, no. 2, p. 24, 2016.

[15] B. A. Rajoub and R. Zwiggelaar, "Thermal facial analysis for deception detection," *IEEE Trans. Inf. Forensics Security*, vol. 9, no. 6, pp. 1015–1023, Jun. 2014.

[16] M. M. Waldrop, "How to hack the hackers: The human side of cybercrime," *Nature*, vol. 533, no. 7602, pp. 164–167, May 2016.

[17] C. Wang, B. Yang, J. Cui, and C. Wang, "Fusing behavioral projection models for identity theft detection in online social networks," *IEEE Trans. Comput. Social Syst.*, vol. 6, no. 4, pp. 637–648, Aug. 2019.

[18] C. Shen, Y. Li, Y. Chen, X. Guan, and R. A. Maxion, "Performance analysis of multi-motion sensor behavior for active smartphone authentication," *IEEE Trans. Inf. Forensics Security*, vol. 13, no. 1, pp. 48–62, Jan. 2018.

[19] C. Wang and H. Zhu, "Representing fine-grained co-occurrences for behavior-based fraud detection in online payment services," *IEEE Trans. Dependable Secure Comput.*, early access, May 4, 2020, doi: 10.1109/TDSC.2020.2991872.

[20] H. Zheng *et al.*, "Smoke screener or straight shooter: Detecting elite sybil attacks in user-review social networks," in *Proc. 25th Annu. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, San Diego, CA, USA, Feb. 2018, pp. 259–300.

[21] R. T. Mercuri, "Scoping identity theft," *Commun. ACM*, vol. 49, no. 5, pp. 17–21, May 2006.

[22] G. Stringhini, P. Mourlanne, G. Jacob, M. Egele, C. Kruegel, and G. Vigna, "EVILCOHORT: Detecting communities of malicious accounts on online services," in *Proc. USENIX Secur.*, 2015, pp. 563–578.

[23] Q. Cao, X. Yang, J. Yu, and C. Palow, "Uncovering large groups of active malicious accounts in online social networks," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2014, pp. 477–488.

[24] G. Stringhini, C. Kruegel, and G. Vigna, "Detecting spammers on social

networks," in *Proc. 26th Annu. Comput. Secur. Appl. Conf. (ACSAC),* 2010, pp. 1–9.

[25] Y. Yao, B. Viswanath, J. Cryan, H. Zheng, and B. Y. Zhao, "Automated crowdturfing attacks and defenses in online review systems," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Dallas, TX, USA, Oct. 2017, pp. 1143–1158.